

WIPRO PYTHON AUTOMATION COURSE

MODULE 4

ROBOT FRAMEWORK

Training & Placement Preparation Report

NAME: - Amit Raj

Course: - Python Automation Testing

Technology: - Robot Framework + Selenium Library + CI/CD

Enrolment No.: - 12023052018002

Stream: - CSBS

Semester: - 7th

Date: - 21/06/2026

Abstract

Robot Framework is an open-source framework that works on the concept of keyword-driven and is used to perform acceptance testing and acceptance test-driven development and many more automation tasks. With SeleniumLibrary, it becomes possible to automate web browsers. The course covers everything from environment set up and basic tests to command line running and browser automation.

The report includes both theoretical content, syntax, architecture, command execution, mini-project, debugging tips, best practices.

Table of Contents

1. Introduction to Robot Framework
 2. Module Scope and Learning Objectives
 3. 3.1 Introduction
 4. 3.2 Script Running Options
 5. 3.3 Make the Script More Readable
 6. Automation Framework Architecture
 7. Practical Mini Project
 8. Execution and Reporting
 9. Troubleshooting and Best Practices
 10. Robot Framework vs Python Test Automation
 11. Conclusion
- Appendix A – Command Sheet

1. Introduction to Robot Framework

Robot Framework is an open-source automation framework where the tests are written using keyword-based syntax that is easy to understand. Keywords used in tests can either be keywords that come prepackaged with the framework, keywords from libraries, or user-defined keywords.

1.1 Why Robot Framework is relevant to Python Automation

Robot Framework has excellent support from the Python community and there are numerous libraries which are built using Python.

Syntax used for keyword makes the test understandable without writing low-level code inside tests.

SeleniumLibrary exposes the browser automation functionality using keywords in Robot.

Resource and User Keywords help in reusability and maintenance.

Command-line based testing suite makes it well suited for batch processing tasks.

1.2 Core Terminology

Term	Meaning
Test Case	A named scenario made from executable keywords and validations.
Keyword	A reusable action supplied by Robot Framework, a library, or the project.
Library	A package exposing keywords to Robot Framework.
Resource File	Reusable .resource file containing keywords, variables and imports.
Suite	A file or directory containing Robot Framework tests.
Variable	Reusable value such as \${URL} or \${BROWSER}.
Tag	Metadata used to categorize or select tests.

Setup/Teardown	Keywords executed before/after tests or suites.
----------------	---

2. Module Scope and Learning Objectives

Section	Topics from supplied outline	Expected competency
3.1 Introduction	Framework intro; setup/packages; RIDE; simple test; keyword/data-driven tests; command line; setup/teardown; Selenium browsers	Create and execute basic Robot web tests.
3.2 Script Running Options	Robot vs Pybot; PyCharm/CMD/batch/Task Scheduler; browser execution; Jenkins; single/multiple suites; advanced options	Execute locally, on schedule and in CI.
3.3 Make Script More Readable	Procedural vs Gherkin; user keywords; breaking scripts into keywords; resource files; setup/teardown; page objects; Gherkin readability	Build reusable and maintainable automation.

2.1 Prerequisites

- Python basics and basic OOP.
- HTML/CSS and browser developer tools.
- Basic Selenium concepts and locators.
- Command-line and pip familiarity.
- Basic testing concepts such as assertions and regression testing.

3. 3.1 Introduction

3.1.1 Introduction to Robot Framework

Robot Framework suites are commonly written in .robot files. Tests are composed from keywords and organized under sections such as Settings, Variables, Test Cases and Keywords.

***** Settings *****

Library String

*** Test Cases ***

Simple Greeting Test

`${text}= Convert To Upper Case hello robot`

`Should Be Equal ${text} HELLO ROBOT`

3.1.2 Environment and Packages

`python --version`

`pip install robotframework`

`pip install robotframework-seleniumlibrary`

`robot --version`

Component	Purpose
Python	Runtime and ecosystem for many Robot libraries/custom libraries.
robotframework	Core Robot Framework package.
SeleniumLibrary	Browser automation keywords based on Selenium.
Browser	Application under test.
RIDE / IDE	Optional editor for Robot Framework files.
Jenkins	CI and scheduled execution.
Git	Version control for suites/resources.

3.1.3 Robot Framework – RIDE

RIDE (Robot Framework IDE) is a GUI editor that has been traditionally used to write tests in Robot Framework. It can edit test suites, keywords and settings. In modern projects, however, it is replaced by VS Code or PyCharm.

3.1.4 Simple Test Case Using RIDE

Test Cases

Addition Should Work

`{result}= Evaluate 10 + 5`

`Should Be Equal As Integers {result} 15`

3.1.5 Keyword-Driven Test Cases

Test Cases

Login With Valid Credentials

Open Browser To Login Page

Enter Username demo_user

Enter Password demo_password

Submit Login

Verify Dashboard Is Visible

Close Browser

The test describes a sequence of reusable actions instead of embedding every implementation detail.

3.1.6 Data-Driven Test Cases

Test Cases

Valid Login - User 1

[Template] Login Should Succeed

user1 password1

Valid Login - User 2

[Template] Login Should Succeed

user2 password2

Keywords

Login Should Succeed

[Arguments] {username} {password}

Log Testing {username}

Should Not Be Empty {password}

Data-driven testing separates test logic from input values and is useful when the same workflow must be checked with many datasets.

3.1.7 Command-Line Execution

robot tests/

robot tests/login.robot

robot -t "Login With Valid Credentials" tests/login.robot

robot -i smoke tests/

robot -e skip tests/

Option	Purpose
robot tests/	Run suites in a directory.
-t	Select tests by name.
-i	Include a tag.
-e	Exclude a tag.
-d	Choose output directory.
-v	Pass a variable.
--loglevel	Control log detail.

3.1.8 Test Setup and Teardown

Settings

Library SeleniumLibrary

Test Setup Open Login Page

Test Teardown Close All Browsers

Test Cases

Valid Login

Input Text id=username demo

Input Password id=password secret

Click Button id=login

Suite Setup/Teardown is used when preparation or cleanup should happen once for a suite. Test Setup/Teardown is used around each test.

3.1.9 Browser Automation with SeleniumLibrary

Settings

Library SeleniumLibrary

Test Cases

Open Example Website

Open Browser https://example.com chrome

Title Should Be Example Domain

Close All Browsers

- Open Browser starts a browser session.

- Input Text/Input Password enter form values.
- Click Button/Click Element performs interactions.
- Element Should Be Visible verifies UI state.
- Close All Browsers performs cleanup.

4. 3.2 Script Running Options

4.1 Robot vs Pybot

Robot vs. Pybot is noted in the syllabus. In today's Robot Framework, the correct command to use is robot. However, there is an old command known as Pybot that you should be aware of.

Term	Meaning
robot	Current standard command-line entry point.
pybot	Legacy command name.
python -m robot	Module-based execution option in suitable environments.

4.2 Running from PyCharm

- Set up the right Python environment/virtual environment.
- Install Robot Framework and its dependencies.
- Open the project and modify .robot files.
- Use the same Robot command you use from the terminal/CI.
- Check output.xml, log.html and report.html files.

4.3 Command Window

```
cd C:\automation\robot_project
robot -d results tests
```

4.4 Batch File

```
@echo off
cd /d C:\automation\robot_project
robot -d results tests
pause
```

4.5 Windows Task Scheduler

The scheduled task will be able to run a batch file at certain times, say every night to perform regression.

- Test Robot command first yourself.
- Add it to the batch file.
- Execute the batch file and view the reports.
- Create a Windows Task Scheduler task.
- Specify working directory, account and conditions.
- See the results post execution.

4.6 Browser Execution with Runtime Variables

robot -d results -v BROWSER:chrome tests/

*****Variables*****

\${BROWSER} chrome

\${BASE_URL} https://example.com

*****Test Cases*****

Open Site

Open Browser \${BASE_URL} \${BROWSER}

[Teardown] Close All Browsers

4.7 Jenkins Integration and Scheduling

Jenkins can check out the automation project, install dependencies, execute Robot tests, and archive the generated result files. A typical flow is:

- Checkout source code.
- Install dependencies.
- Run the Robot command.
- Archive output.xml/log.html/report.html.
- Publish or inspect results and notify the team if required.
- robot -d results tests/

CI/CD principle

The suite should run from a repeatable command without depending on a developer's local IDE. This makes Jenkins integration easier and more reliable.

4.8 Single, Multiple and Advanced Run Options

robot -t "Valid Login" tests/login.robot

robot tests/smoke tests/regression

robot -i smoke tests/

robot -e wip tests/

robot -v ENV:QA tests/

robot --dryrun tests/

robot --loglevel DEBUG tests/

5. 3.3 Make the Script More Readable

Readable automation makes code review, maintenance and debugging easier. Robot Framework supports this through user-defined keywords, variables, resources, setup/teardown, page objects and business-level wording.

5.1 Procedural vs Gherkin-Style

Style	Characteristics
Procedural/keyword sequence	Concrete actions written in execution order.
Gherkin-style	Uses Given/When/Then wording to express behavior.
Hybrid	Business-level keywords backed by normal Robot keywords.

Robot Framework does not require Gherkin. Gherkin-style wording can be used as a readability convention.

5.2 User-Defined Keywords

*****Keywords*****

Login With Credentials

[Arguments] \${username} \${password}

Input Text id=username \${username}

Input Password id=password \${password}

Click Button id=login

Test Cases

Valid Login

Login With Credentials demo secret

Page Should Contain Dashboard

5.3 Break Long Scripts into Keywords

- Open Login Page
 1. Login With Credentials
 2. Submit Login
 3. Verify Dashboard
 4. Search For Product
 5. Add Product To Cart
 6. Complete Checkout
- Reduce duplication.
- Make test cases shorter.
- Centralize changes.
- Improve readability.
- Create reusable business actions.

5.4 Resource Files

resources/login.resource

Settings

Library SeleniumLibrary

Variables

\${USERNAME} id=username

\${PASSWORD} id=password

\${LOGIN_BUTTON} id=login

Keywords

Login With Credentials

[Arguments] \${username} \${password}

Input Text \${USERNAME} \${username}

Input Password \${PASSWORD} \${password}

Click Button \${LOGIN_BUTTON}

tests/login.robot

Settings

Resource ../resources/login.resource

*****Test Cases*****

Valid Login

Login With Credentials demo secret

5.5 Setup and Teardown for Maintainability

*****Settings*****

Library SeleniumLibrary

Suite Setup Open Test Environment

Suite Teardown Close All Browsers

Test Setup Open Login Page

Test Teardown Capture Page Screenshot

5.6 Overview of Page Objects

A page object represents a page or reusable UI component. In Robot Framework, page-object concepts can be implemented with resource files/user keywords or through custom Python libraries/classes.

***** Variables *****

\${USERNAME} id=username

\${PASSWORD} id=password

\${LOGIN_BUTTON} id=login

***** Keywords *****

Enter Login Credentials

[Arguments] \${username} \${password}

Input Text \${USERNAME} \${username}

Input Password \${PASSWORD} \${password}

Submit Login

Click Button \${LOGIN_BUTTON}

5.7 Gherkin-Style Readability

***** Test Cases *****

Successful Login

Given The Login Page Is Open

When The User Logs In With Valid Credentials

Then The Dashboard Should Be Displayed

*** Keywords ***

Given The Login Page Is Open

Go To \${BASE_URL}/login

When The User Logs In With Valid Credentials

Enter Login Credentials demo secret

Submit Login

Then The Dashboard Should Be Displayed

Page Should Contain Dashboard

6. Automation Framework Architecture

robot_project/

├─ tests/

| └─ login.robot

| └─ search.robot

| └─ checkout.robot

├─ resources/

| └─ common.resource

| └─ login.resource

| └─ search.resource

├─ data/

| └─ login.csv

| └─ users.json

├─ libraries/

| └─ custom_library.py

├─ scripts/

| └─ run_smoke.bat

| └─ run_regression.bat

├─ results/

├─ requirements.txt

└─ README.md

Layer	Responsibility
Tests	Business scenarios and assertions.

Resources	Reusable keywords, variables and library imports.
Page objects	Page-specific locators and actions.
Python libraries	Complex/custom functionality.
Data	Input datasets separated from test logic.
Scripts	Repeatable local/scheduled/CI commands.
Results	output.xml, log.html, report.html and evidence.

6.1 Execution Flow

- Robot discovers the selected suites.
- Setup prepares the environment.
- Tests call reusable keywords.
- Keywords interact with the browser through SeleniumLibrary.
- Assertions verify expected behavior.
- Teardown performs cleanup/evidence capture.
- Robot generates results.
- Jenkins/scheduler can archive results.

7. Practical Mini Project

7.1 Project Title

Web Login Automation using Robot Framework + SeleniumLibrary + Reusable Keywords

7.2 Objectives

- Automate a representative login flow.
- Use reusable keywords and a resource file.
- Use setup/teardown.
- Use smoke/regression tags.
- Execute from command line.
- Generate reports suitable for CI.

7.3 Test Scenarios

ID	Scenario	Expected Result
TC01	Valid username/password	Authenticated page appears.
TC02	Invalid username/password	Authentication error appears.
TC03	Valid username/invalid password	Authentication error appears.
TC04	Blank credentials	Required-field validation appears.
TC05	Multiple datasets	Each dataset produces expected behavior.

7.4 Sample Suite

*** Settings ***

Library SeleniumLibrary

Resource ../resources/login.resource

Suite Setup Open Browser To Application

Suite Teardown Close All Browsers

Test Setup Go To Login Page

*** Variables ***

\${BASE_URL} https://example.com/login

\${BROWSER} chrome

*** Test Cases ***

Valid Login

Given The Login Page Is Open

When The User Logs In With Valid Credentials

Then The Dashboard Should Be Displayed

[Tags] smoke regression

Invalid Login

Enter Login Credentials invalid_user wrong_password

Submit Login

Page Should Contain Invalid

[Tags] regression

*** Keywords ***

Open Browser To Application

Open Browser \${BASE_URL} \${BROWSER}

Maximize Browser Window

Go To Login Page

Go To \${BASE_URL}

Given The Login Page Is Open

Go To Login Page

When The User Logs In With Valid Credentials

Enter Login Credentials demo secret

Submit Login

Then The Dashboard Should Be Displayed

Page Should Contain Dashboard

7.5 Execution

robot -d results tests/

robot -i smoke -d results tests/

robot -t "Valid Login" -d results tests/login.robot

8. Execution and Reporting

Robot Framework commonly produces structured and HTML artifacts that support debugging and CI reporting.

Artifact	Purpose
output.xml	Structured execution result for processing/integration.
log.html	Detailed keyword-level execution information.
report.html	High-level test/suite execution summary.
Screenshots	Visual failure evidence when configured.

8.1 Results Directory

robot -d results tests/

8.2 Failure Analysis Process

- Open the report and identify the failed test.
- Inspect the failed keyword and error message.
- Check screenshots or other evidence.
- Classify the cause: application, locator, synchronization, data or environment.
- Fix the correct layer and rerun the smallest relevant test.
- Run the broader suite after validation.

9. Troubleshooting and Best Practices

Problem	Likely cause	Better approach
Element not found	Wrong locator/page not ready	Verify locator and use condition-based waits.
Intermittent failure	Timing/race condition	Improve synchronization; avoid arbitrary sleeps.
Browser fails to start	Environment/browser issue	Check browser compatibility and setup.
Works locally, fails in Jenkins	Environment/config difference	Use reproducible dependencies and explicit configuration.
Very long .robot file	Low-level steps repeated	Extract user keywords/resources.
Duplicated locators	UI details repeated	Centralize page locators.
Unclear reports	Poor names/tags	Use descriptive names and consistent tags.

9.1 Readability Rules

- Use meaningful test names.
- Prefer business-level keywords in tests.
- Keep locators out of tests where practical.
- Use resource files for shared implementation.
- Use tags consistently.
- Keep setup/teardown focused on lifecycle.

- Keep important assertions visible.

9.2 Synchronization

Browser automation is asynchronous. Prefer SeleniumLibrary wait keywords and meaningful conditions rather than fixed sleeps. Condition-based synchronization generally makes suites more predictable and faster.

10. Robot Framework vs Python Test Automation

Aspect	Robot Framework	Python/PyTest-style automation
Test syntax	Keyword/tabular syntax	Python syntax
Readability	Strong for business-level scenarios	Depends on code design
Reuse	User keywords/resource files/libraries	Functions/classes/fixtures/modules
Browser automation	Often via SeleniumLibrary	Often via Selenium directly
Custom logic	Keywords + Python libraries	Native Python
CI execution	Command line/Jenkins friendly	Command line/Jenkins friendly
Main learning focus	Keywords, resources, suites, tags, libraries	Python programming/framework design

These approaches can coexist. Robot Framework can call Python libraries, while Python frameworks can implement similar concepts. Selection depends on project and team requirements.

13. Conclusion

Module 3.0 develops practical Robot Framework skills from basic test creation to browser automation, multiple execution methods and maintainable framework design. The most important progression is from writing steps directly to designing reusable keywords, resource files, page objects and CI-ready execution.

For placement preparation, learners should be able to build a small working suite, execute it from the command line, generate reports, explain its architecture and troubleshoot common

Appendix A – Quick Command Sheet

Command	Purpose
robot --version	Check Robot Framework version.
robot tests/	Run suites in a directory.
robot tests/login.robot	Run one suite.
robot -t "Login Test" tests/	Run one test.
robot -i smoke tests/	Run smoke-tagged tests.
robot -e wip tests/	Exclude wip tests.
robot -d results tests/	Write results to results/.
robot -v BROWSER:chrome tests/	Pass runtime variable.
robot --loglevel DEBUG tests/	Increase log detail.
robot --dryrun tests/	Validate structure without real execution.